

Wollo University
College of Medicine and Health
Sciences

Department of Health Informatics

OOP with Java

By: Shimels D.

Chapter 1:

Introduction to Object-Oriented Programming (OOP)

Lesson Objectives

- At the end of this session students should be able to:
 - Understand the basic concepts and advantages of OOP
 - Compare and contrast procedural programming paradigm and OOP paradigm
 - Explain the basics of java
 - Understand java source code, byte code, compiler, interpreter, JVM and object code
 - Describe Classes, Objects, Class Members and member visibility

Review Exercises

1. What is software?
2. List the types of computer software?
3. What is program?
4. What is programming?
5. What is programming paradigm?
6. Describe the common programming paradigms?
7. What is programming language? List down common programming languages?
8. Discuss the following terms:
 - Source code
 - Byte code
 - Machine code / object code
 - Interpreter
 - Compiler
 - Assembler

Overview of OOP

- Object-Oriented Programming(OOP) is a paradigm to design a program using classes and objects.
- It is the dominant programming paradigm these days, having replaced the structured and procedural programming techniques.
- The object-oriented paradigm couples data and methods together into objects.
- Actions(methods) are defined along with the data in objects in OOP paradigm .

Overview of OOP...

- Structured programming uses **top-down** program design:
 - ✓ To solve a large problem, break the problem into several pieces and work on each piece separately;
 - ✓ To solve each piece, treat it as a new problem that can itself be broken down into smaller problems;
 - ✓ Repeat the process with each new piece until each can be solved directly, without further decomposition.

Overview of OOP...

- Software design using the object-oriented paradigm focuses on objects and operations on objects.
- The object-oriented approach combines the power of the procedural paradigm with an added dimension that integrates data with operations into objects.
- The object-oriented programming approach organizes programs in a way that mirrors the real world, in which all objects are associated with both attributes and activities.

Overview of OOP...

- Using objects improves software reusability and makes programs easier to develop and maintain.
- One of the central issues in software development is how to reuse code.
- Object-oriented programming provides great **flexibility, modularity, clarity, and reusability** through **encapsulation**, **inheritance**, and **polymorphism**.

Overview of OOP...

- The OOP approach to software engineering is to begin by:
 - ✓ **identifying the objects** involved in a **problem**;
 - ✓ **identifying the messages** that those objects should respond to.
- The solution that results is a collection of objects, each with its own data and its own set of responsibilities.
- The objects interact by sending messages to each other(i.e. by method call).

Object-Oriented Programming Languages

- Several object-oriented programming languages have been invented since 1960.
- Among them, **C++**, **Java**, and **C#** are the three most commercially successful OOP languages.
- Inventors and the various OOP languages are given in table 1.1.

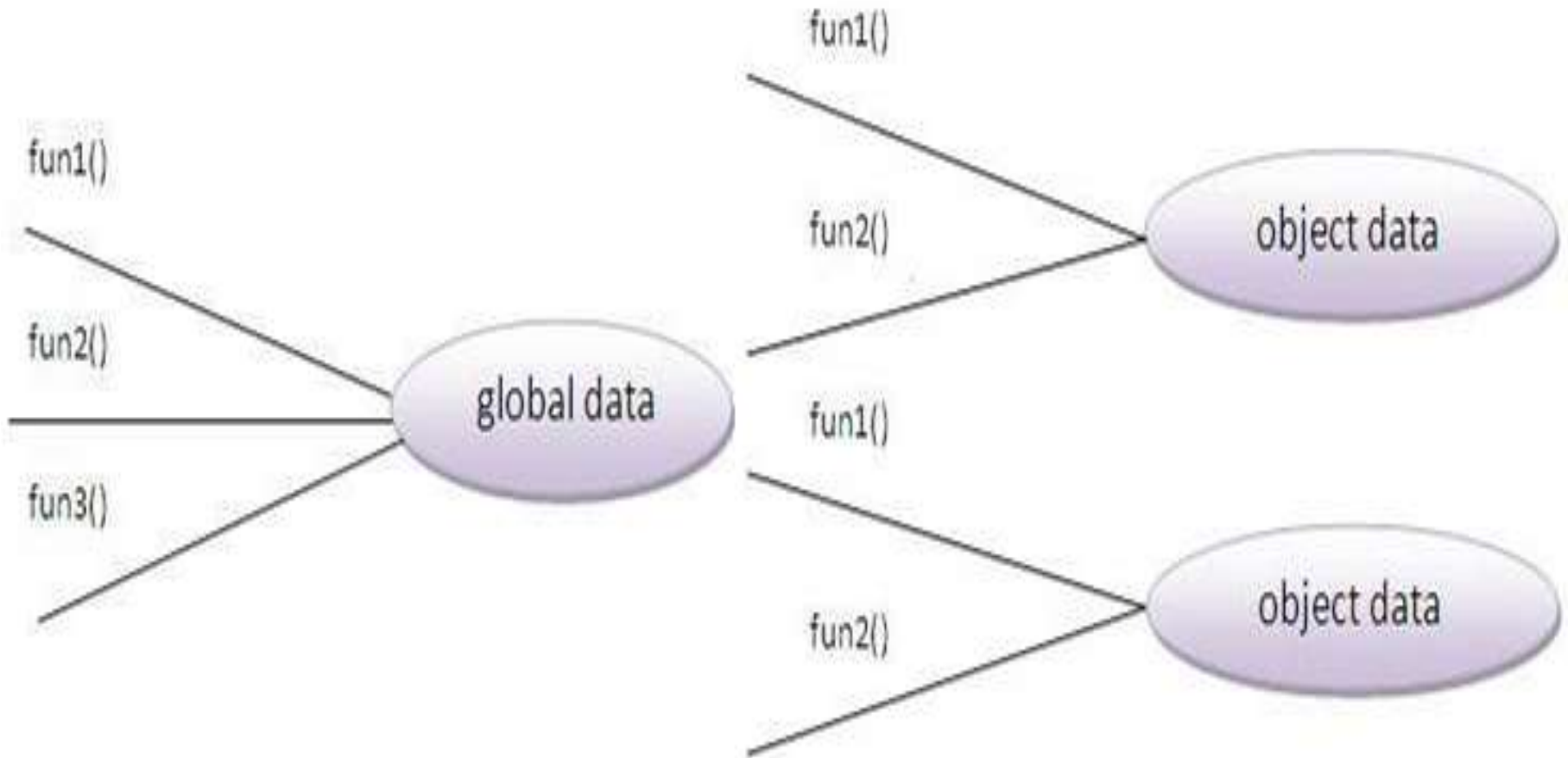
Table 1.1: OOP Languages and their inventors

Language	Inventor, Year	Organization
Simula	Kristen Nygaard and Ole-Johan Dahl, 1960	Defense Research Establishment, Norway
Ada	Jean Ichbiah, 1970	Honeywell-CII-Bull, France
Smalltalk	Alan Kay, 1970	Xerox PARC, USA
C++	Bjarne Stroustrup, 1980	AT&T Bell Labs, USA
Objective C	Brad Cox, 1980	Stepstone, USA
Object Pascal	Larry Tesler, 1985	Apple Computer, USA
Eiffel	Bertrand Meyer, 1992	Eiffel Software, USA
Java	James Gosling, 1996	Sun Microsystems, USA
C#	Anders Hejlsberg, 2000	Microsoft, USA

Advantages of Object-Oriented Programming

- The following are the advantages of software developed using object-oriented programming:
 - Software **reuse** is enhanced.
 - Software maintenance **cost can be reduced**.
 - Data access is restricted **providing better data security**.
 - Software is **easily developed** for complex problems.
 - Software has **improved performance**.
 - Software **quality is improved**.
 - Class hierarchies are helpful in the design process allowing **increased extensibility**.
 - **Modularity** is achieved.
 - **Data abstraction** is possible.

Cont'd...



Limitations of Object-Oriented Programming

- The benefits of OOP may be realized after a long period.
- ✓ Requires intensive testing procedures.
- ✓ Solving a problem using OOP approach consumes more time than the time taken by structured programming approach.

Java – Overview

- Java is a powerful and versatile programming language for developing software or applications running on mobile devices, desktop computers, and servers.
- Java has become enormously popular.
- Its rapid rise and wide acceptance can be traced to its design characteristics, particularly its promise that you can write a program once and run it anywhere -- **Write Once, Run Anywhere (WORA)**.

Application and Applets

- There are two types of programs that can be built in Java: Applications and applets.
- Applications can be directly executed by a JVM. In fact, Java can be used to develop programs for all kinds of applications.
- Applets are small Java programs developed for Internet Applications.
- An applet located on distant computer (Server) can be downloaded via Internet and executed on a local computer (Client) using a Java enabled browser.
- We can develop applets for doing everything from simple animated graphics to complex games and utilities.
- applets are embedded in an HTML document and run inside a Web Page.

Application and Applets...

- Stand alone program can read and write files and perform certain operations that applet can not do. An Applet can only run within a Web Browser.
- The Web browser includes a JVM that provides an execution environment for the applet.
- It is also possible to use a tool called the appletviewer **to run an applet**. This utility is included in the Java Development Kit(JDK) and is used to test applets.
- In this manner, an applet written by any developer in the world may be dynamically downloaded from the Web Server and executed on a client PC or workstation.

Types of Java Applications

- The most common applications that can be created using java programming are:

1) Standalone Application:

- It is also known as **desktop application** or window-based application.
- An application that we need to install on every machine such as media player, antivirus etc.
- Standalone applications are usually created by AWT and Swing components.

2) Mobile Application:

- An application that is created for mobile devices.
- Currently Android and Java ME are used for creating mobile applications.

Types of Java Applications...

3) Web Application:

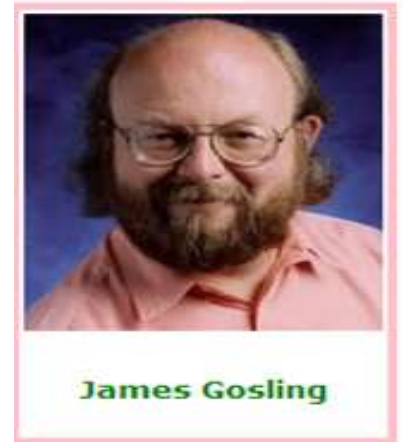
- An application that runs on the server side and creates dynamic page, is called web application.
- Currently servlet, jsp, struts, jsf etc. technologies are used for creating web applications in java.

4) Enterprise Application:

- An application that is distributed in nature, such as banking applications.
- It has the advantage of high level security, load balancing and clustering.
- In java, EJB is used for creating enterprise applications.

Java's History

- James Gosling at Sun Microsystems
- Oak, 1991
- Java, May 20, 1995, Sun World
- Sun Microsystems was purchased by Oracle in 2010.
- Early History Website:



<http://www.java.com/en/javahistory/index.jsp>

Why Java?

- The answer is that Java enables users to develop and deploy applications on the Internet for servers, desktop computers, small hand-held devices, and consumer electronic devices like TVs, VCRs, toasters and such other electronic machines.
- The future of computing is being profoundly influenced by the Internet, and Java promises to remain a big part of that future.
- Java is the Internet programming language.
- Java is a general purpose programming language.

Characteristics of Java

➤ The common features of java are:

- Simple
- Object-Oriented
- Distributed
- Compiled and Interpreted
- Robust
- Secure
- Platform Independent
- Architecture-Neutral
- Portable
- High Performance
- Multithreaded
- Dynamic

Characteristics of Java...

Java is Simple:

- Java is partially modeled on C++, but greatly simplified and improved. Some people refer to Java as "C++--" because it is like C++ but with more functionality and fewer negative aspects.

Java is Object-Oriented:

- Java is inherently object-oriented. Although many object-oriented languages began strictly as procedural languages, Java was designed from the start to be object-oriented.

Characteristics of Java...

Java is Distributed:

- Distributed computing involves several computers working together on a network. Java is designed to make distributed computing easy.
- Since networking capability is inherently integrated into Java, writing network programs is like sending and receiving data to and from a file.

Java is Compiled and Interpreted:

- You need an interpreter to run Java programs. The programs are compiled into the Java Virtual Machine code called bytecode.
- The bytecode is machine-independent and can run on any machine that has a Java interpreter, which is part of the Java Virtual Machine (JVM).

Characteristics of Java...

Java is Robust:

- Java compilers can detect many problems that would first show up at execution time in other languages.
- Java has eliminated certain types of error-prone programming constructs found in other languages.
- Java has a runtime exception-handling feature to provide programming support for robustness.

Java is Secure:

- Java implements several security mechanisms to protect your system against harm caused by stray programs.

Characteristics of Java...

Java is Platform independent:

- Unlike many other programming languages including C and C++, when Java is compiled, it is not compiled into platform specific machine, rather into platform independent byte code.
- This byte code is distributed over the web and interpreted by virtual Machine (JVM) on whichever platform it is being run.

Characteristics of Java...

Java is Architecture-Neutral:

- Write once, run anywhere
- With a Java Virtual Machine (JVM), you can write one program that will run on any platform.

Java is Portable:

- Because Java is architecture neutral, Java programs are portable. They can be run on any platform without being recompiled.

Java's Performance:

- Java is faster than traditional interpretation since byte code is "close" to native code still somewhat slower than a compiled language (e.g., C++).

Characteristics of Java...

Java is Multithreaded:

- Multithread programming is smoothly integrated in Java, whereas in other languages you have to call procedures specific to the operating system to enable multithreading.

Java is Dynamic:

- Java was designed to adapt to an evolving environment.
- New code can be loaded on the fly without recompilation.
- There is no need for developers to create, and for users to install, major new software versions. New features can be incorporated transparently as needed.

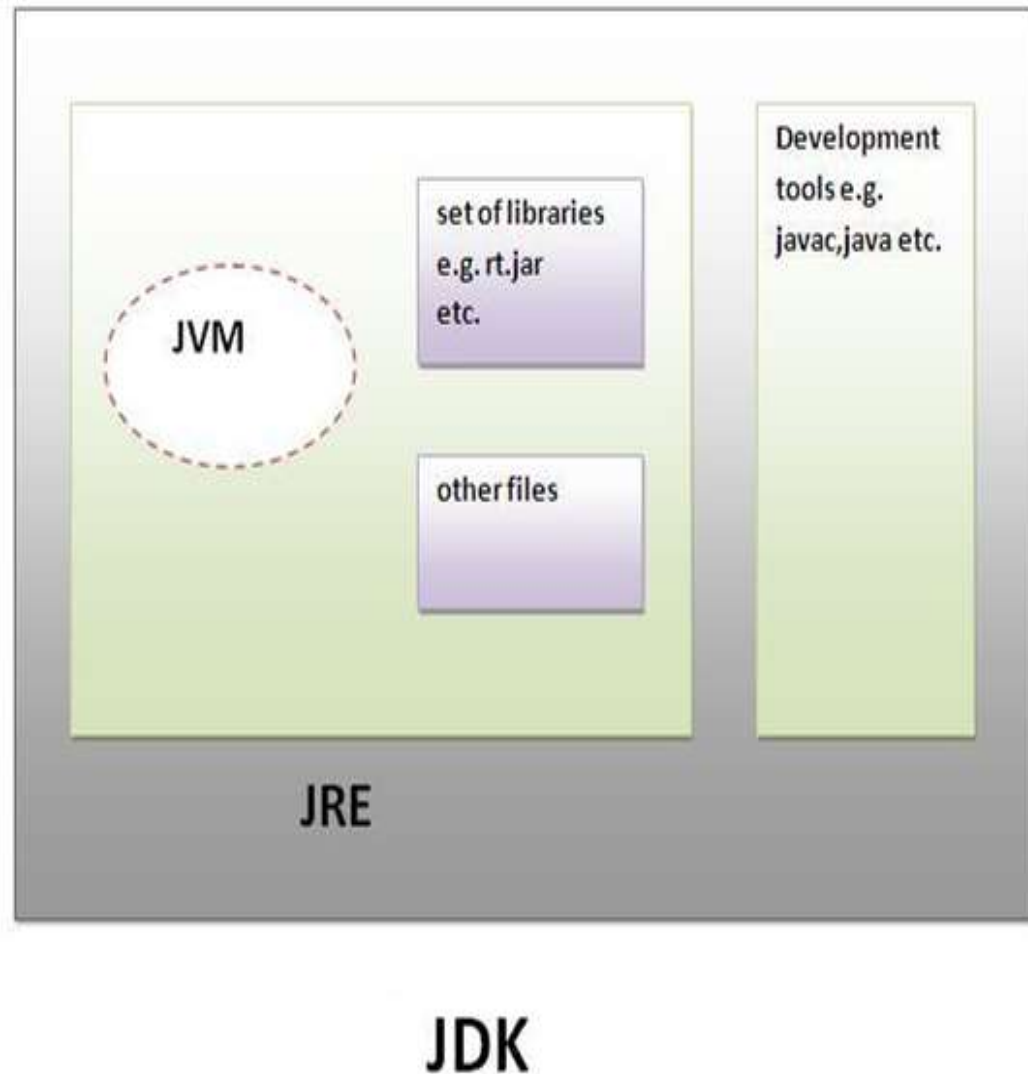
Java Environment and Tools

- Java Environment includes a large number of development tools and hundreds of classes and methods.
- The development tools are part of the system known as Java Development Kit (JDK).
- The classes and methods are part of the Java Standard Library (JSL), also known as the Application Programming Interface (API).
- Common tools you need:
 - ✓ Operating system
 - ✓ Java JDK
 - ✓ IDE
 - ✓ Microsoft Notepad or any other text editor

JDK, JRE and JVM

JDK:

- Java Development Kit
- It physically exists.
- The JDK consists of a set of separate programs (JRE + development tools), each invoked from a command line, for developing , running and testing Java programs.



JDK...

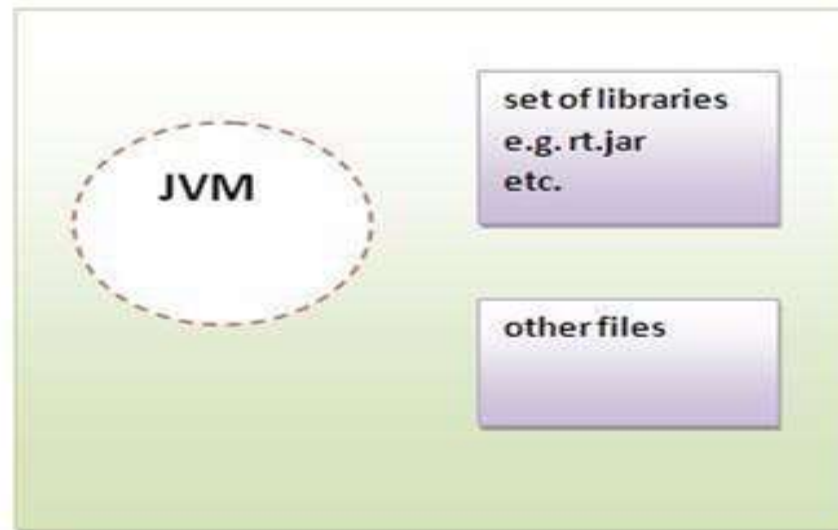
- **JDK** comes with a collection of tools including:
- ✓ **appletviewer** ? Enables us to run Java Applets (Without using java compatible browser).
- ✓ **Java** ? Java Interpreter, which runs applets and applications by reading and interpreting bytecode files.
- ✓ **Javac** ? The Java compiler, which translates Java source code to byte code files that the interpreter understand.
- ✓ **Javadoc** ? Creates HTML-format documentation from Java source code files.
- ✓ **Javah** ? Produces header files for use with native methods.
- ✓ **javap** ? Java disassembler which displays information about the fields, constructors and methods present in a class file.
- ✓ **Jdb** ? Java command line debugger, which helps us to find errors in our programs.
- ✓ ...

JDK Editions

- Java Standard Edition (J2SE)
 - J2SE can be used to develop client-side standalone applications or applets.
 - Java Enterprise Edition (J2EE)
 - J2EE can be used to develop server-side applications such as Java servlets, Java ServerPages, and Java ServerFaces.
 - Java Micro Edition (J2ME).
 - J2ME can be used to develop applications for mobile devices such as cell phones.
- You can download the latest version of JDK SE from:
<http://www.oracle.com/technetwork/java/javase/downloads>)

JRE(Java Runtime Environment)

- is used to provide runtime environment.
- It is the implementation of JVM.
- It physically exists.
- It contains set of libraries + other files that JVM uses at runtime.



JRE

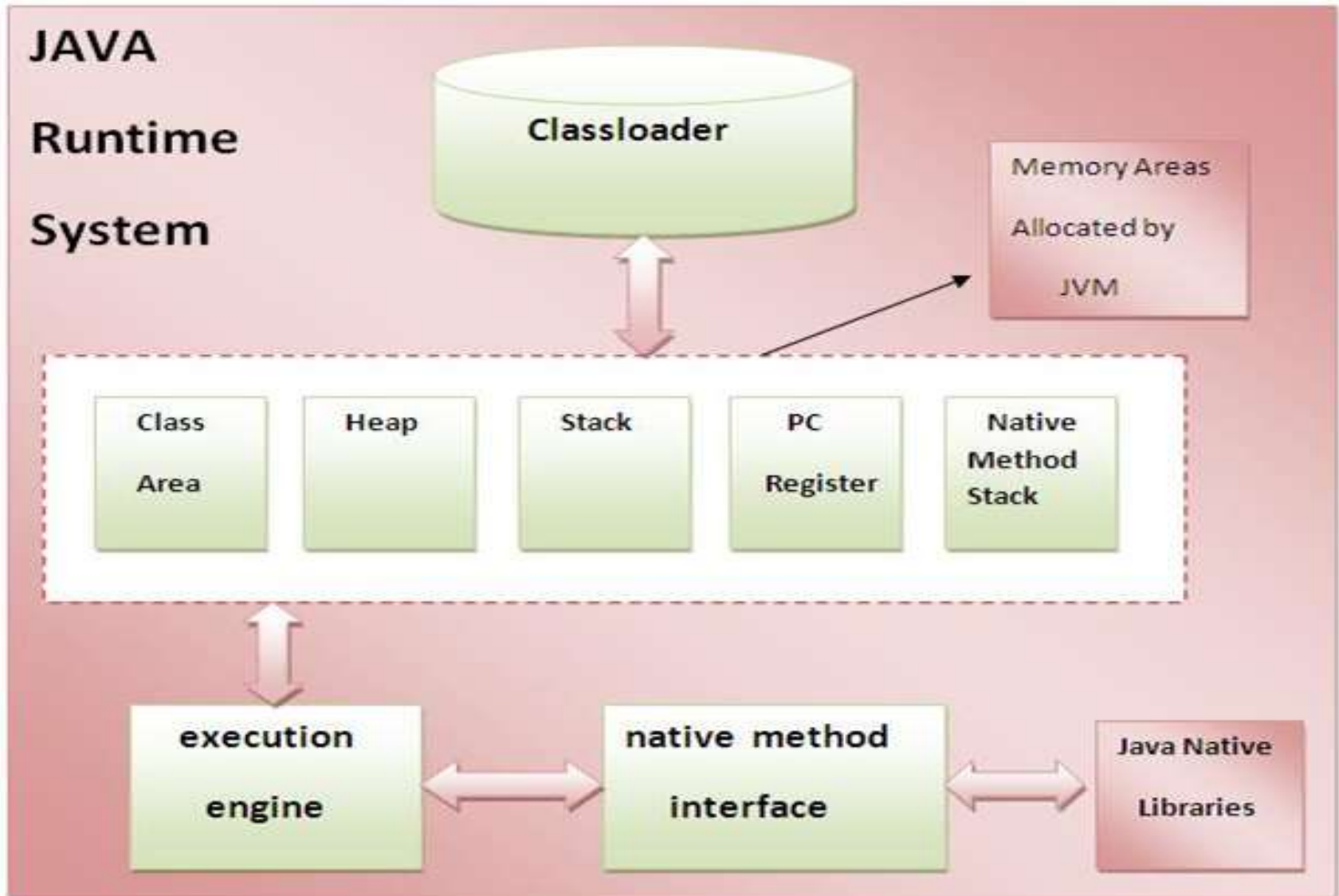
The JVM and Byte Code

- A virtual machine (VM) is a software application layer that simulates a computer but hides the underlying operating system and hardware from the programs that interact with it.
- JVM (Java Virtual Machine) is an abstract machine.
- It is a specification that provides runtime environment in which java bytecode can be executed.
- JVMs are available for many hardware and software platforms.
- Its implementation is known as JRE (Java Runtime Environment).

Cont'd...

- JVM performs the following main tasks:
 - ✓ Loads code
 - ✓ Verifies code
 - ✓ Executes code
 - ✓ Provides runtime environment
- JVM, JRE and JDK are platform dependent because configuration of each OS differs.
- But, Java is platform independent.

Internal Architecture of JVM



Internal Architecture of JVM...

1) Classloader:

- ✓ Classloader is a subsystem of JVM that is used to load class files.

2) Class(Method) Area:

- ✓ Stores per-class structures such as the runtime constant pool, field and method data, the code for methods.

3) Heap:

- ✓ It is the runtime data area in which objects are allocated.

Internal Architecture of JVM...

4) Stack:

- ✓ Java Stack stores frames.
- ✓ It holds local variables and partial results, and plays a part in method invocation and return.
- ✓ Each thread has a private JVM stack, created at the same time as thread.
- ✓ A new frame is created each time a method is invoked.
- ✓ A frame is destroyed when its method invocation completes.

5) Program Counter (PC) Register:

- ✓ It contains the address of the Java virtual machine instruction currently being executed (returnAddress or native pointer).

6) Native Method Stack:

- ✓ It contains all the native methods used in the application.

Internal Architecture of JVM...

7) Execution Engine: a mechanism responsible for executing the instructions contained in the methods of loaded classes.

➤ It contains:

- a) A virtual processor:** assigned to a virtual machine
- b) Interpreter:** Read bytecode stream then execute the instructions.
- c) Just-In-Time(JIT) compiler:** JIT compiles parts of the byte code that have similar functionality at the same time, and hence reduces the amount of time needed for compilation. It translates instruction set of a Java virtual machine (JVM) to the instruction set of a specific CPU.

IDE

- Provides an integrated development environment (IDE) for developing Java programs quickly.
- Editing, compiling, building, debugging, and online help are integrated in one graphical user interface.
- Popular IDEs:
 - NetBeans (<http://www.netbeans.org/index.html>)
 - Eclipse (<http://www.eclipse.org/>)
 - IntelliJ IDEA (<http://www.jetbrains.com/idea/download>)

Interpreting and Compiling Source Code

- A program written in a high-level language is called a *source program* or *source code*.
- Because a computer cannot understand a source program, a source program must be translated into machine code for execution.
- The translation can be done using another programming tool called an *interpreter* or a *compiler*.

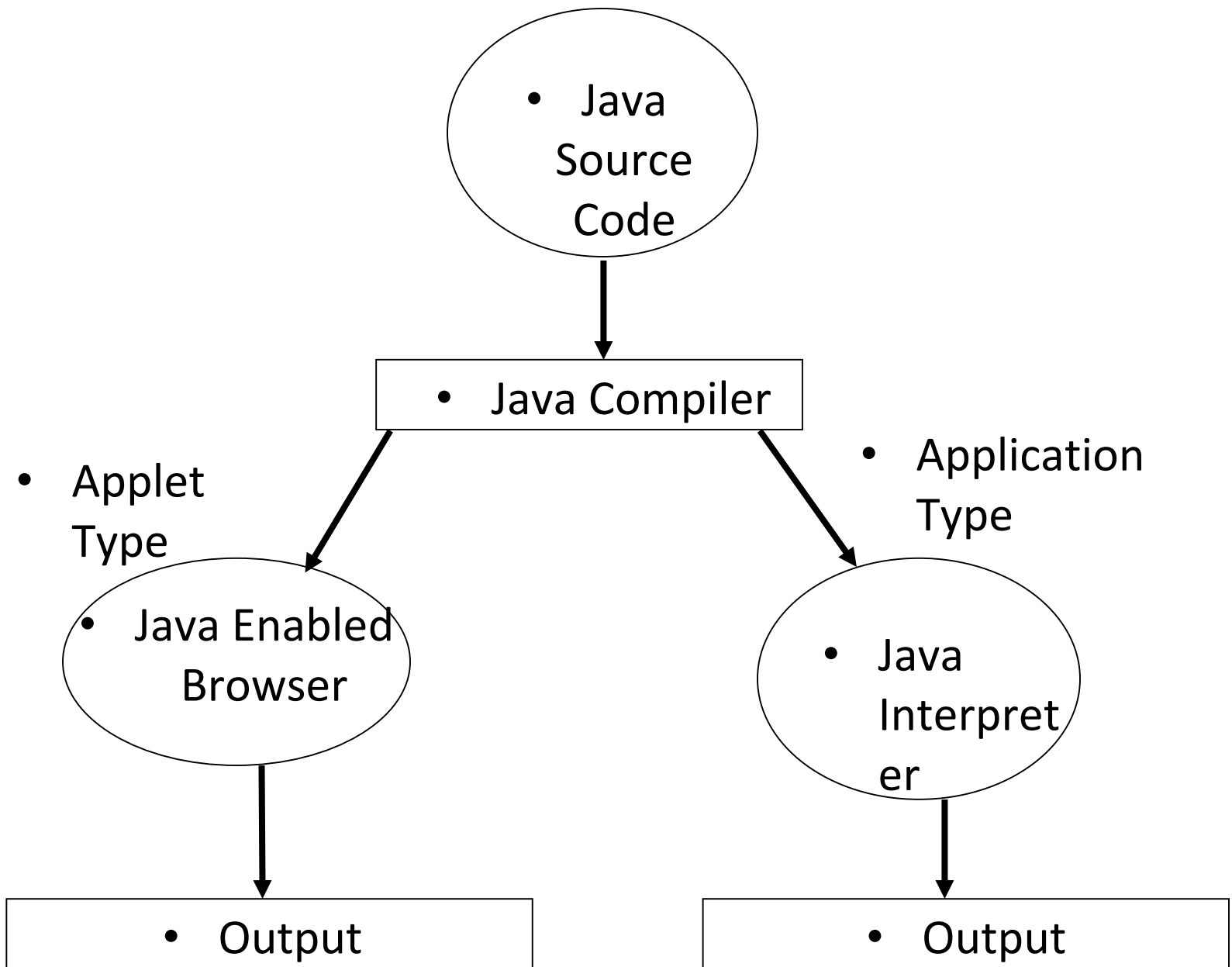
Compiling and Interpreting Source Code

- A compiler translates the entire source code into a virtual machine code file.
- An interpreter reads one statement from the source code, translates it to the machine code or virtual machine code, and then executes it.
- Note that a statement from the source code may be translated into several machine instructions.

Compiling and Interpreting...



Process of converting bytecode into machine code/object code



Java Class Libraries / API

- You can create each class and method you need to form your Java programs.
- However, most Java programmers take advantage of the rich collections of existing classes ,interfaces and methods in the **Java class libraries**, which are also known as the Java **APIs** (Application Programming Interfaces).
- Visit the java SE8 API online documentation:
<http://docs.oracle.com/javase/8/docs/api/index.html>

Java Environment Setup

- Once you downloaded and installed Java on your machine, you would need to set **environment variables(binary files)** of JDK to point to correct installation directories.
- Environment variable **PATH** should be set to point to where the Java binaries have been installed.
- The **PATH** is the system variable that your operating system uses to locate needed executables from the command line or terminal window.

Steps to Create and Execute Java App

- We now explain the commonly used steps in creating and executing a Java application using a Java development environment.
- Java programs normally go through five phases:-
 1. **edit,**
 2. **compile,**
 3. **load,**
 4. **verify and**
 5. **execute.**
- Let's discuss these phases in the context of the Java SE Development Kit (**JDK**).

Phase 1: Editing / Creating a Program

- You type a Java program (typically referred to as **source code**) using the editor or IDE, make any necessary corrections and save the program.

Example:

//Simple java source code

```
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to java  
programming!");  
    }  
}
```


Phase 2: Compiling a Java Program into Bytecodes

- In Phase 2, you can use the command **javac** (the Java compiler) to compile a program.

javac Welcome.java

- NetBeans IDE typically provides a menu item, such as Compile File(F9) to compile the program.
- If the program compiles, the compiler produces a **.class** file called **Welcome.class** that contains the compiled version of the program.
- The Java compiler translates Java **source code** into **bytecodes** that represent the tasks to execute in the execution phase (Phase 5).

Phase 2...

- **Bytecodes** are executed by the **Java Virtual Machine (JVM)**—a part of the JDK and the foundation of the Java platform.
- bytecodes are platform independent—they do not depend on a particular hardware platform.
- So, Java's bytecodes are portable—without recompiling the source code, the same bytecodes can be executed on any platform containing a JVM.
- The JVM is invoked by the java command.

java Welcome

- NetBeans IDE typically provides a menu item, such as Run(SHIFT + F6), that invokes the java command for you.
- This begins Phase 3.

Phase 3: Loading a Program into Memory

- In Phase 3, the JVM places the program in memory to execute it—this is known as loading.
- The JVM's class loader takes the **.class** files containing the program's bytecodes and transfers them to primary memory.
- The class loader also loads any of the **.class** files provided by Java that your program uses.
- The **.class** files can be loaded from a disk(rt.jar) on your system or over a network (e.g., your local college or company network, or the Internet).

Phase 4: Bytecode Verification

- In Phase 4, as the classes are loaded, the bytecode verifier examines their bytecodes to ensure that they're valid and do not violate Java's security restrictions.
- Java enforces strong security to make sure that Java programs arriving over the network do not damage your files or your system (as computer viruses and worms might).

Phase 5: Execution

- In Phase 5, the JVM's execution engine executes the program's bytecodes, thus performing the actions specified by the program.
- Translates bytecodes into object code.

Thank You

Next chapter: Overview of Java

